Lab 17: Decision Tree Classifier Analysis

Implementing Decision Tree Classifier and analyzing the effect of tree depth on performance.

```
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 1: Load the dataset and encode the target variable diagnosis (M=1, B=0).

```
# Load the dataset
df = pd.read_csv('/content/data.csv')

# Display the first few rows and check info to understand the data
print("Dataset Head:")
display(df.head())
print("\nDataset Info:")
df.info()

# Encode the target variable 'diagnosis' (M=1, B=0)
# 'M' for Malignant, 'B' for Benign
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

print("\nEncoded 'diagnosis' column head:")
display(df[['diagnosis']].head())
```

```
Dataset Head:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_m
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1

5 rows × 33 columns

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB

Encoded 'diagnosis' column head:
```

	diagnosis
0	1
1	1
2	1
3	1
4	1

Step 2: Split the dataset into training and testing sets (80/20).

```
# Drop unnecessary columns. The 'id' column is an identifier and 'Unnamed: 32' is likely empty.
X = df.drop(columns=['id', 'diagnosis', 'Unnamed: 32'])
y = df['diagnosis']

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Training features shape: {X_train.shape}")
print(f"Testing features shape: {X_test.shape}")
print(f"Training target shape: {y_train.shape}")
print(f"Testing target shape: {y_test.shape}")
```

```
Training features shape: (455, 30)
Testing features shape: (114, 30)
Training target shape: (455,)
Testing target shape: (114,)
```

Step 3 & 4: Train Decision Tree Classifier with varying `max_depth` values (3, 5, 7, 10) and evaluate models.

```
max_depth_values = [3, 5, 7, 10]
results = {
    'max_depth': [],
    'accuracy': [],
    'precision': [],
    'recall': [],
    'f1_score': []
}

for depth in max_depth_values:
    print(f"\n--- Training Decision Tree with max_depth = {depth} ---")

    # Initialize Decision Tree Classifier
    dt_classifier = DecisionTreeClassifier(max_depth=depth, random_state=42)

    # Train the model
    dt_classifier.fit(X_train, y_train)

    # Make predictions on the test set
    y_pred = dt_classifier.predict(X_test)

    # Evaluate the model
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # Store results
    results['max_depth'].append(depth)
    results['accuracy'].append(accuracy)
    results['precision'].append(precision)
    results['recall'].append(recall)
    results['f1_score'].append(f1)

    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1-Score: {f1:.4f}")
    print("Classification Report:")
    print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.97      0.93        72
           1       0.94      0.79      0.86        42

    accuracy                           0.90       114
   macro avg       0.91      0.88      0.89       114
weighted avg       0.91      0.90      0.90       114


--- Training Decision Tree with max_depth = 5 ---
Accuracy: 0.9211
Precision: 0.9459
Recall: 0.8333
F1-Score: 0.8861
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.97      0.94        72
           1       0.95      0.83      0.89        42

    accuracy                           0.92       114
   macro avg       0.93      0.90      0.91       114
weighted avg       0.92      0.92      0.92       114


--- Training Decision Tree with max_depth = 7 ---
Accuracy: 0.9386
```

```
           0       0.93      0.97      0.95        72
           1       0.95      0.88      0.91        42

    accuracy                           0.94       114
   macro avg       0.94      0.93      0.93       114
weighted avg       0.94      0.94      0.94       114


--- Training Decision Tree with max_depth = 10 ---
Accuracy: 0.9298
Precision: 0.9048
Recall: 0.9048
F1-Score: 0.9048
Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.94      0.94        72
           1       0.90      0.90      0.90        42

    accuracy                           0.93       114
   macro avg       0.92      0.92      0.92       114
weighted avg       0.93      0.93      0.93       114
```

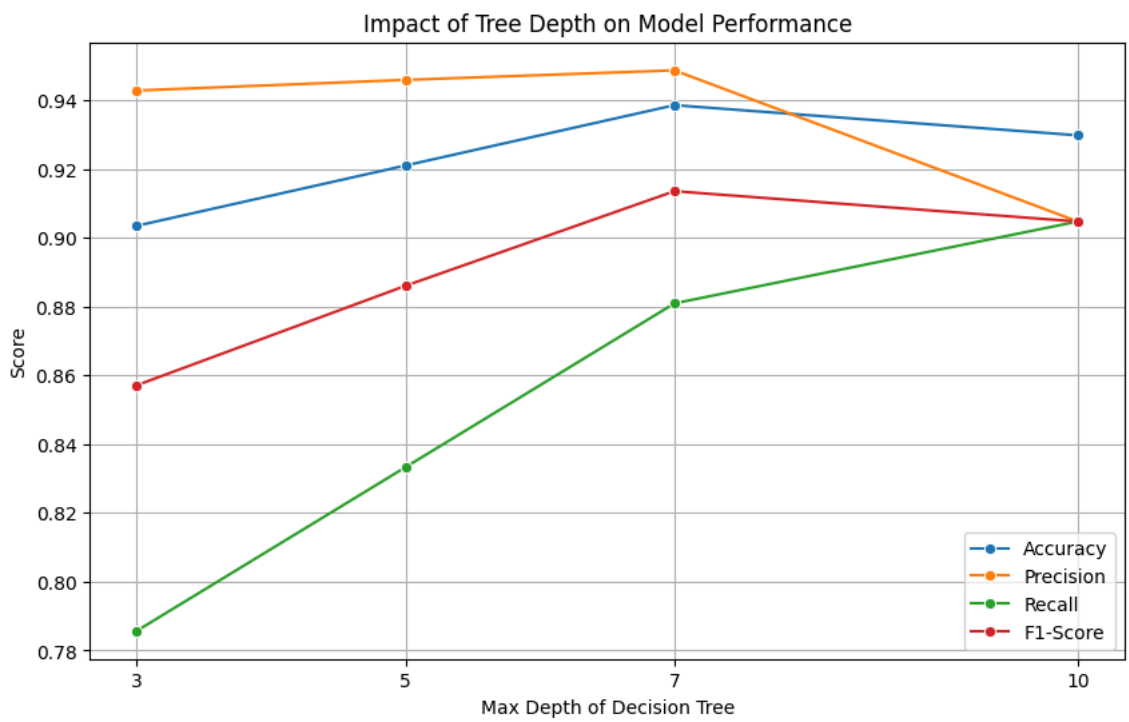
Visualize Model Performance

```
# Convert results dictionary to a pandas DataFrame for easier plotting
results_df = pd.DataFrame(results)

plt.figure(figsize=(10, 6))
sns.lineplot(x='max_depth', y='accuracy', data=results_df, marker='o', label='Accuracy')
sns.lineplot(x='max_depth', y='precision', data=results_df, marker='o', label='Precision')
sns.lineplot(x='max_depth', y='recall', data=results_df, marker='o', label='Recall')
sns.lineplot(x='max_depth', y='f1_score', data=results_df, marker='o', label='F1-Score')

plt.title('Impact of Tree Depth on Model Performance')
plt.xlabel('Max Depth of Decision Tree')
plt.ylabel('Score')
plt.xticks(max_depth_values)
plt.grid(True)
plt.legend()
plt.show()

display(results_df)
```



	max_depth	accuracy	precision	recall	f1_score
0	3	0.903509	0.942857	0.785714	0.857143
1	5	0.921053	0.945946	0.833333	0.886076
2	7	0.938596	0.948718	0.880952	0.913580
3	10	0.929825	0.904762	0.904762	0.904762